

Parte 1 Introducción a la ingeniería de software 1

Capítulo 1 Introducción

- 1.1 Desarrollo de software profesional
- 1.2 Ética en la ingeniería de software
- 1.3 Estudios de caso

Objetivos

Los objetivos de este capítulo consisten en introducir al lector a la ingeniería de software y ofrecer un marco conceptual para entender el resto del libro. Al estudiar este capítulo:

- Conocerá qué es la ingeniería de software y por qué es importante;
- Comprenderá que el desarrollo de diferentes tipos de sistemas de software puede requerir distintas técnicas de ingeniería de software;
- Entenderá algunos conflictos éticos y profesionales que son importantes para los ingenieros de software;
- Conocerá tres sistemas de diferentes tipos, que se usarán como ejemplos a lo largo del libro.

PUNTOS CLAVE

- La ingeniería de software es una disciplina de ingeniería que se interesa por todos los aspectos de la producción de software.
- El software no es sólo un programa o programas, sino que también incluye documentación. Los atributos esenciales de los productos de software son mantenimiento, confiabilidad, seguridad, eficiencia y aceptabilidad.
- El proceso de software incluye todas las actividades que intervienen en el desarrollo de software. Las actividades de alto nivel de especificación, desarrollo, validación y evolución son parte de todos los procesos de software.
- Las nociones fundamentales de la ingeniería de software son universalmente aplicables a todos los tipos de desarrollo de sistema. Dichos fundamentos incluyen procesos, confiabilidad, seguridad, requerimientos y reutilización de software.
- Existen muchos tipos diferentes de sistemas y cada uno requiere para su desarrollo de herramientas y técnicas adecuadas de ingeniería de software. Existen pocas, si es que hay alguna, técnicas específicas de diseño e implementación que son aplicables a todos los tipos de sistemas.
- Las ideas fundamentales de la ingeniería de software son aplicables a todos los tipos de sistemas de software. Dichos fundamentos incluyen procesos de administración de software, confiabilidad y seguridad del software, ingeniería de requerimientos y reutilización de software.
- Los ingenieros de software tienen responsabilidades con la profesión de ingeniería y la sociedad. No deben preocuparse únicamente por temas técnicos.
- Las sociedades profesionales publican códigos de conducta que establecen los estándares de comportamiento esperados de sus miembros.

Bibliografía:

INGENIERÍA DE SOFTWARE, Novena edición, Ian Sommerville

El desarrollo de la Guía de Estudio 1 es en forma Grupal.

Debe efectuar la lectura y análisis del Capítulo 1 y luego resolver los siguientes ejercicios.

No se aceptarán copias textuales en el desarrollo de la presente Guía de Estudio.

CONSIGNAS

1. Naturaleza del software profesional

- a. Sommerville afirma que el software no es solo código: incluye documentación, pruebas, requisitos, y mantenimiento. EXPLICAR POR QUÉ es importante considerar estos elementos como parte integral del producto.

Actividad: BUSCAR un ejemplo de software real (puede ser libre o comercial) e IDENTIFICAR qué elementos (además del código) lo acompañan.

- b. REFLEXIONAR sobre las diferencias entre software desarrollado de forma profesional y software “aficionado” (por hobby, sin procesos formales).

¿QUÉ implicancias tiene esto en términos de confiabilidad, mantenimiento y seguridad?

¿PUEDE un software aficionado volverse profesional?

2. Tipos y modelos de desarrollo de software

- a. Sommerville distingue entre **software genérico** (comercializado a múltiples usuarios) y **software personalizado** (hecho a medida). EXPLICAR esta diferencia con ejemplos (p. ej.: MS Excel vs. sistema de gestión hospitalaria).

- b. ¿CÓMO influye esta diferencia en la forma de desarrollar, probar y mantener el software?

¿QUÉ ventajas y desventajas tiene cada tipo para desarrolladores y usuarios?

3. Atributos de calidad del software

- a. Sommerville destaca la importancia de atributos como **mantenibilidad**, **confiabilidad**, **eficiencia** y **aceptabilidad**. DEFINIR cada uno y ARGUMENTAR POR QUÉ son clave en cualquier proyecto.

- b. PROPONER otros atributos relevantes para determinados contextos, como: portabilidad, escalabilidad, accesibilidad, usabilidad. EXPLICAR con ejemplos por qué son importantes hoy.

4. Desafíos actuales y futuros de la disciplina

- a. INVESTIGAR e IDENTIFICAR tres desafíos históricos en la ingeniería de software: por ejemplo, el **incremento de la complejidad**, la **falta de calidad en los entregables**, o los **plazos mal estimados**. EXPLICAR CÓMO impactaron en proyectos reales.

- b. IDENTIFICAR dos desafíos emergentes del siglo XXI. Sugerencia: CONSIDERAR temas como sostenibilidad, inteligencia artificial, privacidad, trabajo remoto, cambio climático o automatización.

5. Diversidad de sistemas y enfoques de desarrollo

Existen muchos tipos de sistemas: embebidos, empresariales, interactivos, de tiempo real, distribuidos.

- a. SELECCIONAR dos tipos diferentes y ANALIZAR QUÉ particularidades tienen en su diseño y desarrollo.

- b. ¿QUÉ herramientas o enfoques específicos se usan en cada caso?

¿QUÉ pasaría si se aplicaran técnicas genéricas a un sistema especializado?

6. Fundamentos universales de la ingeniería de software

- a. Sommerville plantea ideas aplicables a todo tipo de desarrollo (por ejemplo: ingeniería de requisitos, gestión de procesos, validación). EXPLICAR brevemente al menos tres de estas ideas fundamentales.

- b. ¿POR QUÉ son necesarias independientemente del tipo de sistema o tecnología usada?

- c. ESTABLECER vínculos entre estas ideas y los desafíos discutidos en el punto 4.

7. Transformaciones por la Web

- a. DESCRIBIR CÓMO era el software empresarial antes de la Web (monolítico, local, cerrado).
- b. ¿QUÉ nuevas posibilidades aparecieron con la Web (p. ej., acceso remoto, servicios en la nube, SaaS)?
- c. EXPLICAR CÓMO la Web transformó el desarrollo de software empresarial.

Actividad: BUSCAR un software que haya evolucionado antes y después de la Web (p. ej.: facturación, encuestas, gestión educativa).

8. Profesionalización y ética en la disciplina

- a. ¿CONSIDERAR que debería exigirse certificación para ser ingeniero/a de software, como en otras profesiones reguladas? FUNDAMENTAR desde el punto de vista del impacto social del software.
- b. ¿QUÉ implica actuar éticamente como desarrollador/a? DAR ejemplos de dilemas reales o hipotéticos (por ejemplo, una app que recolecta más datos de los necesarios).

9. Aplicación del Código de Ética ACM/IEEE

SELECCIONAR al menos tres cláusulas del código ético presentado en el libro y PROPONER una situación concreta (de la vida real o hipotética) en la que aplicar esa cláusula ayude a tomar una mejor decisión.

Ejemplo: “Evitar daño” – No lanzar una app con fallas conocidas que puedan comprometer datos médicos.

10. Ética y vigilancia digital

Frente al crecimiento de sistemas de vigilancia masiva y uso de datos personales para seguridad nacional:

- a. ¿ACEPTARÍA trabajar en un proyecto de este tipo? ¿QUÉ condiciones pondrían?
- b. ¿ES POSIBLE proteger la privacidad y al mismo tiempo aportar a la seguridad?

Actividad: BUSCAR un caso real (p. ej., China, EE.UU., Europa) sobre sistemas de vigilancia y ANALIZAR sus consecuencias éticas y sociales.

EVALUACIÓN

Criterio	Puntaje	Descripción
Análisis del Material Bibliográfico	2 puntos	Evaluación de la comprensión profunda del texto. El estudiante debe ser capaz de extraer los conceptos clave y relacionarlos con ejemplos o situaciones prácticas.
Integración Coherente de Conceptos	2 puntos	Los estudiantes deben aplicar los conceptos de la bibliografía de manera coherente en las respuestas, demostrando la capacidad de integrar teoría y práctica.
Información Precisa y Completa	2 puntos	Respuestas precisas y detalladas, no copiadas textualmente. Se valorará que el estudiante sea capaz de formular sus propias respuestas basadas en los textos.
Orden y Estética Adecuada	1 punto	Se evaluará la estructura de las respuestas: claridad en la organización, uso adecuado de referencias, imágenes, gráficos, y fuentes bien citadas.
Participación de los Integrantes del Grupo	1 punto	Evaluación de la colaboración activa de cada miembro del grupo en el trabajo. Se considera la equidad en la distribución de tareas y aportes al desarrollo del trabajo.
Uso Complementario de IA	2 puntos	Se permite el uso de IA de manera complementaria para consultar conceptos o expandir ideas, pero no sustituye el análisis personal. El uso debe ser mínimo.

Uso de la IA: Consideraciones

- El 80% del puntaje se centra en la aplicación de conceptos, la comprensión crítica y la expresión autónoma de las ideas por parte del estudiante.
- El 20% restante permite el uso de IA como apoyo complementario. Esto significa que la IA puede ser utilizada para consultar definiciones, expandir ideas o resolver dudas específicas, pero no debe sustituir la reflexión y el razonamiento propio.

Indicaciones para el Uso de IA:

1. IA como herramienta de apoyo: El uso de IA debe ser complementario. Los estudiantes pueden consultar y expandir conceptos o aclarar dudas específicas, pero la respuesta final debe estar basada en su propio razonamiento.
2. No se permite el uso de IA para generar respuestas completas o resolver ejercicios de manera automática. Las respuestas deben ser únicas y reflexivas.
3. Transparencia: Si se utiliza IA, el estudiante debe explicar brevemente cómo la utilizó y mostrar las ideas generadas por la IA, asegurándose de que las respuestas sean propias y no completamente generadas por herramientas automáticas.
4. Reflexión personal: Se espera que el estudiante defienda sus respuestas y sea capaz de explicar cómo construyó su respuesta, incluso si utilizó IA en la consulta de un concepto.